

PushWorld: Optimizing Learned Search

Inference-time optimizations for a frozen transformer policy

Evgeny Ivankin, Adam Suliman

Training setup and evaluation target

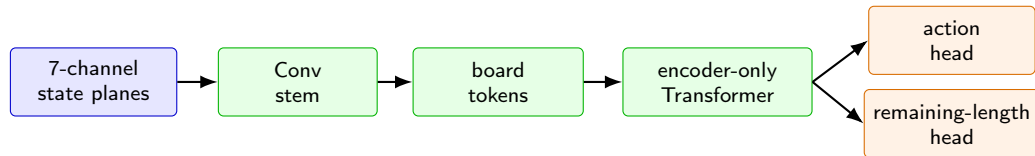
Policy training

- Expert traces generated by the official C++ N+RGD planner.
- Imitation learning on roughly half of all Level-0 puzzle families.
- No Level-1 training signal in the checkpoint used for these search experiments.
- Model weights are fixed during all optimizations in this presentation.

Evaluation

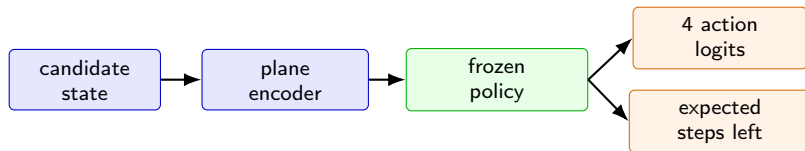
- 68 original Level-1 puzzles.
- Maximum 100 actions per puzzle.
- Metrics: solved puzzles, wall-clock time, solves/minute, model evaluations.
- Same checkpoint, different inference/search strategy.

Model architecture



- Input is the current symbolic state encoded as planes: walls, agent, movable objects, goals.
- The policy is memoryless at inference: no trajectory history is passed to the model.
- The action head proposes moves; the remaining-length head is used as a value-like estimate for search.

Data flow for one model query



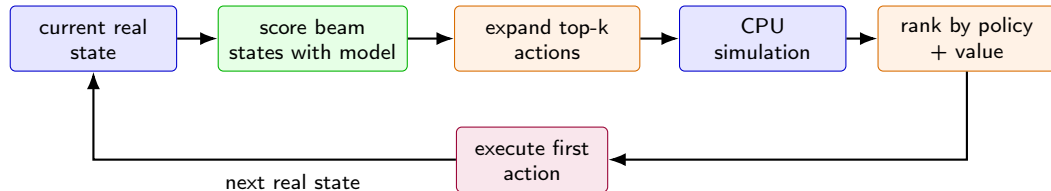
Action logits

- rank the four primitive moves;
- used for greedy rollout, beam expansion, and CEM sampling.

Remaining-length head

- predicts a discretized estimate of steps left;
- guides value-aware beam and best-first search.

Learned beam search loop



- Model scoring runs on GPU; candidate-state simulation and search bookkeeping run on CPU.
- The same state can appear in many neighboring replans.
- Expensive part: repeated model queries over overlapping search trees, not the four-action branching itself.

Main bottlenecks before optimization

Where time goes

- Beam replans after every real action.
- Overlapping lookahead trees repeatedly score the same states.
- GPU model calls are mixed with CPU search and simulation.
- CPU/GPU transfers are not dominant alone; redundant model work is.

Measured signal

- Beam without cache: 660k model-forwarded states.
- Beam with cache: 28.7k unique states.
- Same success rate: 18/68.
- Cache changes runtime, not solver behavior.

Reference solver: N+RGD

- N+RGD is the official DeepMind C++ planner based on recursive graph-distance heuristics.
- It is the strongest reference we use: solves all Level-1 puzzles very quickly.
- It is not optimized for shortest plans; on puzzles solved by the learned policy, our returned path is always equal or shorter in the measured comparison.
- For this project, N+RGD is a reference solver and trace generator, not the system we optimize.

Method	Solved	Total time	Avg / puzzle	Solves/min
N+RGD C++ planner	68/68	6.7s	0.10s	607.9
Learned beam, no cache	18/68	1699.6s	25.0s	0.64
Learned beam, cache	18/68	98.2s	1.44s	11.0

Optimization 1: prediction cache

Target bottleneck

- repeated model scoring for identical (*puzzle*, *state*);
- especially strong in closed-loop beam search.

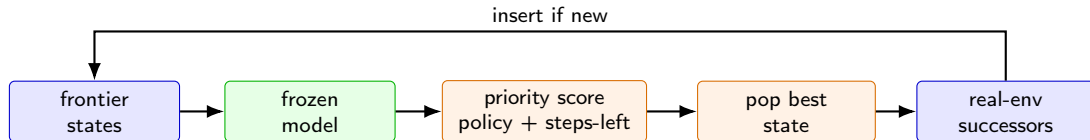
Implementation

- cache key: puzzle id + symbolic state;
- cache value: action log-probs and expected remaining length;
- no change in action selection logic.

Beam setting	Solved	Time	Relative speed
no prediction cache	18/68	1699.6s	1.0x
prediction cache	18/68	98.2s	17.3x

Same solved set, fewer model forwards: 660k $\rightarrow$ 28.7k.

Optimization 2: value-guided best-first search



- Beam spends a fixed budget at every depth; best-first spends the node budget on globally promising states.
- Priority combines policy cost and the remaining-length head.
- It improves search quality without changing model weights.

Best-first results

Search method	Solved	Total time	Avg / puzzle	Solves/min
Beam + cache	18/68	98.2s	1.44s	11.0
Best-first, budget 512	20/68	119.7s	1.76s	10.0
Best-first, budget 1024	32/68	199.6s	2.94s	9.62

- Main quality gain: 18/68 $\rightarrow$ 32/68 on the same frozen policy.
- Cost increases because best-first explores more unique states.
- Budget 1024 is the best learned solver by success rate.

Optimization 3: GPU sampling

- Instead of expanding a deterministic search tree, sample several policy rollouts and verify them in the real environment.
- This gives a speed-oriented upper bound for stochastic policy use.
- The current model is still the limiting factor: sampling is fast, but many sampled rollouts collapse or repeat.

Method	Solved	Total time	Avg / puzzle	Solves/min
GPU sampling, best-of-4	16/68	48.0s	0.71s	20.0
Beam + cache	18/68	98.2s	1.44s	11.0

Fastest learned inference mode, but success rate is below beam and best-first.

Optimization 4: CEM sampling

Algorithm

- 1 Sample many action sequences from the policy.
 - 2 Replay each sequence in the real PushWorld simulator.
 - 3 Keep elite rollouts by solved/progress score.
 - 4 Update a per-timestep action prior and resample.
- CEM improves over simple sampling, but runtime grows quickly.
 - It is an exploratory alternative, not the final best tradeoff.

CEM setting	Solved	Time	Avg/puzzle	Solves/min
16x3	22/68	137s	2.02s	9.61
32x3	24/68	216s	3.17s	6.67
64x3	25/68	332s	4.88s	4.52

Optimization 5: GPU-resident particle rollout

- Goal: move candidate rollout dynamics to GPU instead of calling the Python simulator for every candidate.
- Exact backend: one fused Triton kernel for PushWorld transition.
- Approximate backend: point-object dynamics, faster but not always faithful.
- Particle count means how many candidate trajectories are simulated in parallel for one puzzle.
- Result: exact GPU transition works, but end-to-end search is still dominated by policy refresh and verification.

Variant	Scale	Smoke result	Verdict
Beam + cache	5 puzzles	1/5 in 7.5s	comparison run
Exact Triton particles	128 particles	1/5 in 68s	too slow/weak
Exact Triton particles	512 particles	2/5 in 238s	more success, too slow
Approx dynamics	128 particles	false positives	needs exact replay filter

Ranker experiment

- We tested a hand-written ranker term inside best-first priority.
- No additional model was trained: priority used policy cost, remaining-length estimate, goal distance, and achieved-goal bonus.
- This was a cheap proxy for learned state ranking.
- Result: no measurable gain over baseline best-first 1024.

Search priority	Solved	Total time	Avg / puzzle	Solves/min
policy + remaining length	32/68	199.6s	2.94s	9.62
+ goal-distance ranker term	32/68	199.8s	2.94s	9.61

Final comparison

Method	Solved	Time	Avg / puzzle	Solves/min	Role
N+RGD planner	68/68	6.7s	0.10s	607.9	reference solver
Beam, no cache	18/68	1699.6s	25.0s	0.64	bottleneck baseline
GPU sampling	16/68	48.0s	0.71s	20.0	fastest learned mode
Beam + cache	18/68	98.2s	1.44s	11.0	cached baseline
CEM 16x3	22/68	137.3s	2.02s	9.61	stochastic search
Best-first 1024	32/68	199.6s	2.94s	9.62	best learned success

- Final choice for quality: **prediction cache + best-first 1024**.
- Final choice for raw speed: GPU sampling, but with lower success.

① **Learned ranker for search nodes.**

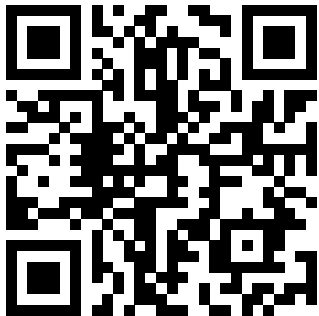
Train a lightweight ranker on states expanded by best-first: solution-path states as positives, dead/repeated frontier states as negatives. Use it to reduce node budget without losing solved puzzles.

② **Adaptive search budget.**

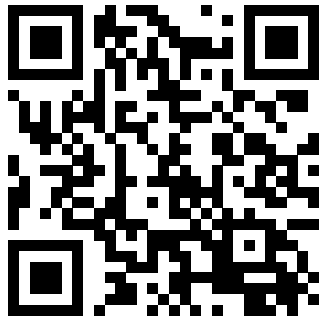
Start with cheap sampling or BF-512, then escalate only when confidence/progress is low. This targets better average time per puzzle instead of one fixed budget for all maps.

③ **Proposal + verification split.**

Use fast approximate or GPU proposals only to generate candidate action sequences, then always verify the final path in exact PushWorld dynamics.



github.com/eivankin/pushworld



github.com/adam-suliman/pushworld